

Signals

Part II

Sistemas de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Recall from last class

- **A signal is a notification of an event sent to a specific process**
 - Process receives an integer that maps to a specific event
 - They are a limited form of inter-process communication
- **Notifications are asynchronous**
 - A process may receive a notification at any time, during its execution
- **If you can't predict when an event occurs, then it is not practical to handle it in the main body of the program**
 - The solution is to write specialized code to handle the signal and execute it only when necessary

Safe signal handling

- **Handlers are tricky because they are concurrent with main program and share the same global data structures**
 - Shared data structures can become corrupted
 - We'll explore concurrency issues later in the semester
- **Keep your handlers as simple as possible**
 - All substantial work should be deferred to the main execution path, where normal synchronization mechanisms are safe to use
- **Call only async-signal-safe functions in your handlers**
 - `printf`, `sprintf`, `malloc`, and `exit` are not safe!
 - Explore the manual pages to view a list of signal-safe functions (`man signal-safety`)

Safe signal handling

- **Protect accesses to shared data structures by temporarily blocking all signals to prevent possible corruption**
- **Declare global variables as `volatile` to prevent the compiler from storing them in a register**
 - Prevents the compiler from optimizing away repeated reads
 - Ensures each check reloads the value from memory
- **Declare global flags as `sig_atomic_t`**
 - It is guaranteed to be read and written atomically
 - Use only in a variable that is only read or written in one operation (e.g., `flag = 1`, not `flag++`)

The safe pattern

■ Design principle: a signal handler should notify, not perform

- The handler communicates that “something happened”
- The main program decides what to do about it — safely

■ In the signal handler:

- Only set a global flag of type `volatile sig_atomic_t`
- Do not call non-async-signal-safe functions, allocate memory, acquire locks, or modify complex shared data structures

■ In the main program flow:

- Periodically check that flag
- Perform the actual work safely in normal execution context
- Use proper synchronization mechanisms if needed

The safe pattern

```
volatile sig_atomic_t terminate = 0;

void handle_INT(int signo){
    /* Inside signal handlers, only async-signal-safe functions should be used */
    write(STDOUT_FILENO, "\nCatch SIGINT!\n", 13);
    terminate = 1;
}

int main(int argc, char *argv[]){
    struct sigaction act;

    /* Zeroes the sigaction structure */
    memset(&act, 0, sizeof(struct sigaction));

    act.sa_handler = handle_INT;
    act.sa_flags = SA_RESTART;
    sigaction(SIGINT, &act, NULL);

    while(!terminate) {
        /* normal work */
    }

    /* cleanup safely here */
}
```

Setting the action for a signal

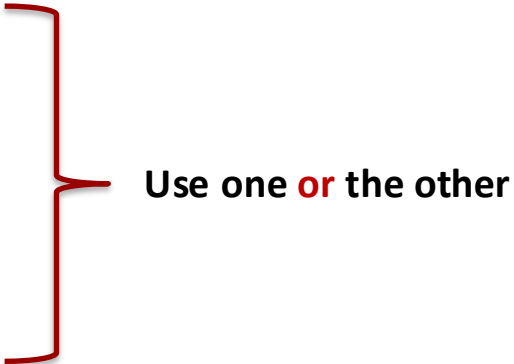
```
#include <signal.h>

int sigaction(int sig,
              const struct sigaction *act,
              struct sigaction *oact);
```

- Last class, we saw that `sigaction()` modifies the disposition of a process when receiving a specific signal `sig`
 - `act` specifies the action details for the signal `sig`
 - `oact` (if not `NULL`) stores the previously set action details

struct sigaction

■ May differ slightly across architectures but these fields must comply with POSIX

- `sa_handler`
 - Pointer to an ANSI C handler function (previous class)
 - `sa_sigaction`
 - Pointer to a POSIX handler function (**this class**)
 - `sa_mask`
 - This specifies a set of signals to be blocked while the handler runs (**this class**)
 - `sa_flags`
 - This specifies various flags which can affect the behavior of the signal (previous class)
- 
- Use one **or** the other

The `sa_sigaction` field

- **Used to set a POSIX-compliant signal handler**
 - Provides **more flexibility and control** over the signal handling process
 - Requires the `SA_SIGINFO` flag to be set in the `sa_flags` field to enable extended information passing to the handler
- **Cannot be used simultaneously with an ANSI C signal handler, as they are mutually exclusive**
- **Possible values:**
 - A pointer to a POSIX signal handler function that is executed upon signal reception

The `sa_sigaction` field

- `void (*sa_sigaction)(int sig, siginfo_t *info, void *ucontext)`
 - `sig`
 - The number of the signal that caused the invocation of the handler
 - `info`
 - A pointer to a `siginfo_t` structure containing further information about the signal
 - `ucontext`
 - A pointer to a `ucontext_t` structure, cast to `void *`, containing signal context information that was saved on the user-space stack by the kernel (rarely used)
- We'll concentrate on the second parameter, the `siginfo_t` structure

The `siginfo_t` struct

- The `sigaction()` man page on Linux contains a definition of this structure
 - Which fields are filled in depends on how the signal is sent, the source of the signal, and the actual signal type

```
siginfo_t {  
    int si_signo;           /* Signal number */  
    int si_errno;          /* An errno value */  
    int si_code;           /* Signal code */  
    int si_trapno;         /* Trap num that caused hardware-generated sig */  
    pid_t si_pid;          /* Sending process ID */  
    uid_t si_uid;          /* Real user ID of sending process */  
    int si_status;         /* Exit value or signal */  
    union sigval si_value; /* Signal value */  
    ...  
    int si_syscall;        /* Number of attempted system call */  
    unsigned int si_arch;  /* Architecture of attempted system call */  
}
```

Example – capturing SIGUSR1

```
volatile sig_atomic_t last_signal = 0;
volatile sig_atomic_t sender_pid  = 0;

void handle_USR1(int signo, siginfo_t *sinfo, void *context){
    write(STDOUT_FILENO, "\nInside handler!\n", 17);
    last_signal = sinfo->signo;
    sender_pid  = sinfo->si_pid;
}

int main(int argc, char *argv[]){
    struct sigaction act;

    /* Zeroes the sigaction structure */
    memset(&act, 0, sizeof(struct sigaction));

    act.sa_sigaction = handle_USR1;
    act.sa_flags = SA_SIGINFO;
    sigaction(SIGUSR1, &act, NULL);

    /* SIGUSR1 will now be captured and handled (POSIX handler) */
    /* (...) */
}
```

Example – setting an alarm

```
volatile sig_atomic_t got_alarm;

void handle_ALARM(int signo, siginfo_t *sinfo, void *context){
    write(STDOUT_FILENO, "\nInside handler!\n", 17);
    got_alarm = 1;
}

int main(){
    int i = 0;
    struct sigaction act;
    got_alarm = 0;

    memset(&act, 0, sizeof(struct sigaction));
    act.sa_sigaction = handle_ALARM;
    act.sa_flags = SA_SIGINFO;
    sigaction(SIGALRM, &act, NULL);

    alarm(5);
    while(!got_alarm){
        printf("%d : Working hard\n", i++);
        sleep(1);
    }
    return 0;
}
```

Takeaway

- The `sigaction` structure provides two alternative ways of defining a signal handler
 - Depending on the level of information and control required

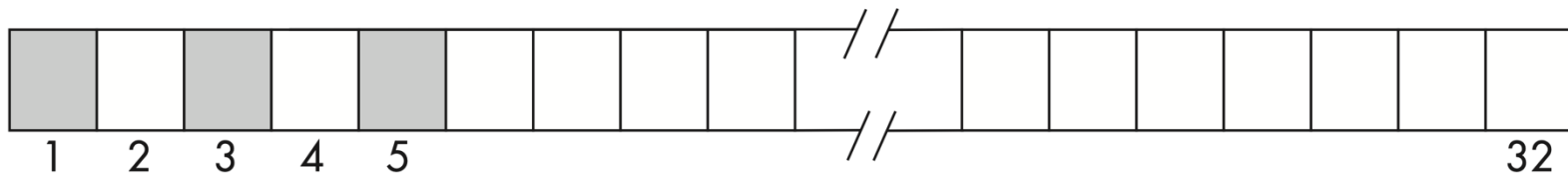
Feature	<code>sa_handler</code>	<code>sa_sigaction</code>
POSIX-compliant	Basic	Full
Extra info (<code>siginfo_t</code>)	✗	✓
Requires <code>SA_SIGINFO</code>	✗	✓
Typical use	Simple handlers	Advanced handling

More advanced signal handling

- **The following slides explore two advanced yet essential concepts in signal handling**
- **Handling multiple signals**
 - Understand the behavior when multiple signals arrive while a handler is being executed
- **Signal blocking**
 - Temporarily preventing specific signals from being delivered

Handling multiple signals

- When a signal is being handled, subsequent notifications **of the same signal number** are automatically **blocked**
- For standard signals, the kernel records only whether a signal is pending, not how many times it occurred
 - Real-time signals behave differently
- We can think of it as a bit mask with a bit for every signal type
 - A signal is blocked if and only if its corresponding bit in the mask is set



Handling multiple signals

■ Once the handler returns

- If the bit is still set, the kernel will deliver the signal again
- But it **does not distinguish how many times the signal arrived** in the meantime

■ Therefore, you can't use signals to count events!

- Such as children terminating

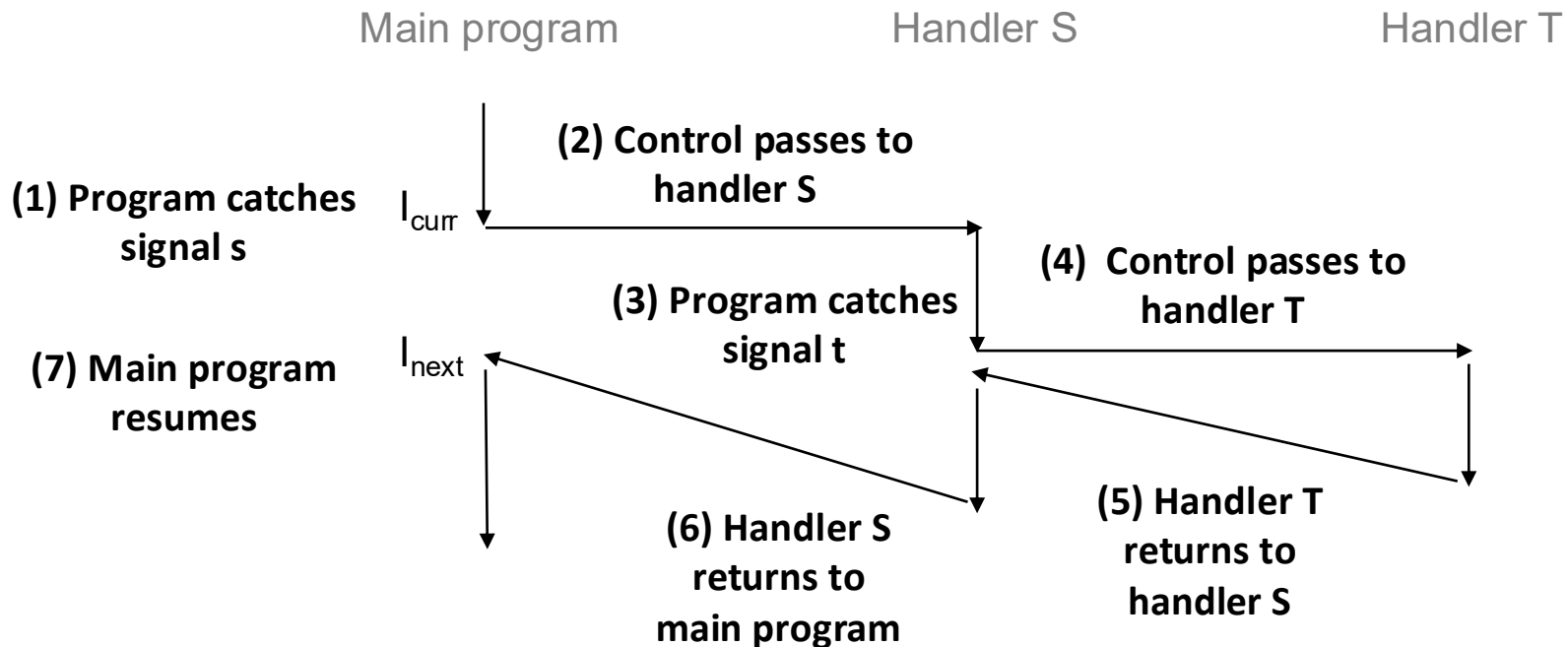
■ But what about other signals?

- By default, they will **interrupt the current handler** to trigger their own handler

Nested signal handlers

■ Handlers can be interrupted by other handlers

- Adds complexity to signal handling logic to avoid issues such as race conditions or deadlocks



Handling multiple signals

- The `sa_mask` field of `struct sigaction` is used to specify which **additional signals should be blocked** during the execution of the signal handling function
 - Not allowing an incoming signal to interrupt the signal handling
- A blocked signal is not ignored!
 - The signal is **delayed until it becomes unblocked**, i.e., when the handler completes
- Signal blocking is implemented using a `sigset_t` data type
 - The set of blocked signals is then manipulated using functions such as `sigaddset`, `sigdelset`, etc.
 - (collectively referred to as `sigset` operations)

Example – adding signals to the set

```
void handle_USR1(int signo, siginfo_t *sinfo, void *context){
    ...
}

int main(int argc, char *argv[]){
    struct sigaction act;

    memset(&act, 0, sizeof(struct sigaction));

    sigemptyset(&act.sa_mask);          /* No signals blocked */
    sigaddset(&act.sa_mask, SIGINT);    /* Block SIGINT */
    sigaddset(&act.sa_mask, SIGUSR2);  /* Block SIGUSR2 */

    act.sa_sigaction = handle_USR1;
    act.sa_flags = SA_SIGINFO;
    sigaction(SIGUSR1, &act, NULL);

    /* Now, when a SIGUSR1 is received, SIGINT and SIGUSR2 are blocked
       during the execution of handle_USR1 */

    /* (...) */
}
```

Example – removing signals from the set

```
void handle_USR2(int signo, siginfo_t *sinfo, void *context){
    ...
}

int main(int argc, char *argv[]){
    struct sigaction act;

    memset(&act, 0, sizeof(struct sigaction));

    sigfillset(&act.sa_mask);           /* All signals blocked */
    sigdelset(&act.sa_mask, SIGCHLD);  /* Unblock SIGCHLD */
    sigdelset(&act.sa_mask, SIGUSR1);  /* Unblock SIGUSR1 */

    act.sa_sigaction = handle_USR2;
    act.sa_flags = SA_SIGINFO;
    sigaction(SIGUSR2, &act, NULL);

    /* Now, when a SIGUSR2 is received, all signals except SIGCHLD and SIGUSR1 are blocked
       during the execution of handle_USR2 */

    /* (...) */
}
```

Blocking/unblocking signals

- **Blocking signals and preventing interrupts to signal handlers are related but distinct concepts**
- **Blocking a signal means the OS is instructed to hold onto the signal and not deliver it to the process immediately**
 - The signal **remains pending**
 - Think of it like putting a message in a queue for later delivery
- **Primarily used to prevent a signal from interrupting a sensitive section of code**
 - This ensures that critical operations (like updating shared data structures) can complete without being disrupted (a topic for future classes)

The `sigprocmask` function

```
#include <signal.h>

int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

- Used to fetch and/or change the **signal mask** of the calling process
 - The signal mask is the set of signals whose delivery is currently blocked for the caller
- The behavior of the call is dependent on the value of `how`
 - `SIG_BLOCK` – The set of blocked signals is the union of the current set and the `set` argument
 - `SIG_UNBLOCK` – The signals in `set` are removed from the current set of blocked signals
 - `SIG_SETMASK` – The set of blocked signals is set to the argument `set`

The `sigpending` function

```
#include <signal.h>

int sigpending(sigset_t *set);
```

- **Fetch the set of signals that are pending for delivery to the calling process**
 - (i.e., the signals which have been raised while blocked)
 - The mask of pending signals is returned in `set`

Important note

■ When a process is created using `fork()`

- The child inherits the signal dispositions of the parent (including any custom handlers set with `sigaction()`)
- The child also inherits the signal mask

■ When a process executes a new program using `exec()`

- All custom signal handlers are reset to their default actions
- Signals set to be ignored remain ignored
- Pending signals are not preserved across `exec()`

■ Key takeaway

- Signal handlers survive `fork()`, but are reset by `exec()`

Example 1 – blocking/unblocking signals

```
int main(){
    sigset_t mask, pending;

    sigemptyset(&mask);
    sigaddset(&mask, SIGINT);
    sigprocmask(SIG_BLOCK, &mask, 0);
    /* CTRL-C is now blocked! */

    sleep(10);

    /* Check if SIGINT is pending... */
    sigpending(&pending);
    if(sigismember(&pending, SIGINT))
        printf("User tried to interrupt the sleep function\n");

    sigprocmask(SIG_UNBLOCK, &mask, 0);
    /* CTRL-C is now unblocked! */

    /* (...) */
}
```

Example 2 – blocking/unblocking signals

- Assume a signal handler sets a `sig_atomic_t` flag, and you need to check the flag before performing an action
- If the signal is delivered right after you check the flag but before the action, it can cause unexpected behavior
 - An example of the check-then-act race condition
- To ensure reliable testing, **check the flag while the signal is blocked**

Example 2 – blocking/unblocking signals

```
volatile sig_atomic_t flag = 0;

void handle_signal(int signum) {
    flag = 1;
}

int main() {
    struct sigaction sa;
    sigset_t block_mask, old_mask;

    sa.sa_handler = handle_signal;
    sa.sa_flags = SA_RESTART;
    sigemptyset(&sa.sa_mask);

    /* At this point, no signals are blocked during handler execution */

    /* Register the signal handler for SIGUSR1 */
    sigaction(SIGUSR1, &sa, NULL);

    ...
}
```

Example 2 – blocking/unblocking signals

```
sigemptyset(&block_mask);
sigaddset(&block_mask, SIGUSR1);

while (1) {
    /* Block the signal */
    sigprocmask(SIG_BLOCK, &block_mask, &old_mask);

    /* Check the flag while the signal is blocked */
    if(flag) {
        perform_action();
        flag = 0;
    }

    /* Unblock the signal (so it can be delivered if it's pending) */
    sigprocmask(SIG_SETMASK, &old_mask, NULL);

    sleep(5);
}

return 0;
}
```

Takeaway

■ Ignoring a signal (disposition)

- The signal is delivered to the process, but it is explicitly discarded by the signal handler
- This is a permanent state (unless the disposition is changed)

■ Blocking a signal (masking)

- The signal is prevented from being delivered
- It remains in a pending state
- The signal is typically held until the process unblocks it (the mask is cleared)
- Once a blocked signal is finally delivered, the process may then choose to execute a handler or ignore it entirely

Exercise 1

■ Child process:

- Executes in a loop, printing messages, handling, and blocking signals
- Creates a handler for `SIGUSR1`, `SIGUSR2`, and `SIGINT`
- Prints a message each time one of these signals is handled
- Ensures the handler is not interrupted by other signals
- On handling `SIGINT`, sets a flag to trigger termination
- Sets up a separate handler for `SIGALRM`
- Blocks `SIGALRM` and prints a message when it is pending

■ Parent process:

- Sends signals to the child process and waits for its termination
- Sends 1 x `SIGALRM`, 3 x `SIGUSR1`, 1 x `SIGUSR2`, 1 x `SIGINT`